

Inigo Quilez :: articles :: sse for cpu tracers - 2005



This article is about SIMD programming and was written in 2005. Things have changed since then, but this is still here for reference. It explains the super basics of doing SSE2 programming for algorithms that need to take branches and might have divergent flow for the different lanes of execution. This is typically the case in a raytracer or raymarcher for example. The article uses a Mandelbrot set renderer as an example to keep things simple, but the principles apply equally.

So, converting your regular raytracing to a parallel model makes branching difficult many times. There are two approaches that can be taken for that parallelization: data parallelization or code parallelization. In the trivial examples on the net, such as normalizing a 3D vector, normally code parallelization is used because it's easier to implement, and normally doesn't suffer from branching difficulties. But as I said, those examples lack from real usability. In real applications, data parallelization is used instead, what means you normally have to change the structure of your code, at least in the low level parts of your design. I will try to give a brief explanation about this data parallelization in this tutorial, and give an example on how to handle conditional branching when using SIMD instructions. The technique explained here can be (and has been) successfully used for speeding up many applications as physics engine or raytracing. I expect you to have basic knowledge of what SSE instructions look like, and how the data is stored in this new 128 bit representation. If you don't know that, don't worry, you will master the subject in less than 15 minutes by reading all those other tutorials (especially if you ever programmed in ASM, C or C++) - I'm not going to rewrite the same thing one more time. So, let's go!

As you know SSE (or AltiVec) technology allows us to compute up to four operations at the same time. This operations normally include arithmetic, logic, memory and comparison operations (plus some other additional special functions). However, in most common cases, you want to use SIMD instructions to make calculations (up to four times) faster. Let's directly go to the example of this tutorial, a 2D Mandelbrot set rendering. Let's use the simplest (and oldest) algorithm used, the well known escape time algorithm. In plain C code, the algorithm looks like this:

```
void drawMandelbrot( uint32 *buffer, int xres, int yres )
{
    const float ixres = 1.0f/(float)xres;
    const float iyres = 1.0f/(float)yres;
```

```

for( int j=0; j < yres; j++ )
for( int i=0; i < xres; i++ )
{
    const float a = -2.25f + 3.00f*(float)i*xres;
    const float b =  1.12f - 2.24f*(float)j*yres;

    *buffer++ = IterateMandelbrot( a, b );
}
}

```

Where we basically loop over the pixels of the framebuffer, and we call the *IterateMandelbrot()* function that will really do the math. That one, could look like this:

```

uint32 IterateMandelbrot( float a, float b )
{
    float x, y, x2, y2;

    x = x2 = 0.0f;
    y = y2 = 0.0f;

    // iterate f(Z) = Z^2 + C,  z0 = 0
    for( int i=0; i<512; i++ )
    {
        y = 2.0f*x*y+b;
        x = x2-y2+a;

        x2 = x*x;
        y2 = y*y;

        const float m2 = x2+y2;
        if( m2>4.0f )
            break;
    }

    // create color
    return 0xff000000 | (i<<16) | (i<<8) | i;
}

```

At this point we are in the classical C scalar code. Now, we will try to speed up the computations by using SSE instructions. Instead of using the code parallelism and try to do some of the multiplications in the inner loop in parallel (what is really difficult and also useless), we will use data parallelization: we will compute four pixels of our image all in parallel. This approach is very useful because you almost don't have to change your algorithm's code that much, as we will see. Well, first we have to change slightly our raster double loop in the *drawMandelbrot()* function. We will advance in each scanline four pixels per iteration, thus calculating four values for the parameter *a* (one per pixel). Let's transform our code to something like this:

```

void drawMandelbrot( uint32 *buffer, int xres, int yres )
{
    __m128i *buffer4 = (__m128i*)buffer;
    const __m128 ixres = _mm_set1_ps( 1.0f/(float)xres );
    const __m128 iyres = _mm_set1_ps( 1.0f/(float)yres );

    for( int j=0; j < yres; j++ )
    for( int i=0; i < xres; i+=4 )
    {
        __m128  a, b;
        a = _mm_set_ps( i+3, i+2, i+1, i+0 );
        a = _mm_mul_ps( a, ixres );
        a = _mm_mul_ps( a, _mm_set1_ps( 3.00f ) );
        a = _mm_add_ps( a, _mm_set1_ps(-2.25f) );

        b = _mm_set1_ps( (float)j );
        b = _mm_mul_ps( b, iyres );
        b = _mm_mul_ps( b, _mm_set1_ps(-2.24f) );
        b = _mm_add_ps( b, _mm_set1_ps( 1.12f) );

        _mm_store_si128( buffer4++, IterateMandelbrot( a, b ) );
    }
}

```

Note that this code can be optimized to perform only a single addition per inner iteration. However we will leave the code like this for more clear understanding of what the code is doing.

Now, we have four different values of **a** and **b** in those variables. Thus the new **IterateMandelbrot()** function will receive the four values of **(a,b)** and output the color for four pixels. Since we are using true color output (32 bit per pixel), the type **__m128i** for returning the 4 pixel colors is just perfect (remember that **__m128** is used for four floating point values, while **__m128i** is used for four 32 bit integer values).

So, the new **IterateMandelbrot()** can be easily translated into SSE. For that, we change the normal scalar multiplications with the vectorial version, and same holds for addition and subtraction:

```

__m128i IterateMandelbrot( __m128 a, __m128 b )
{
    __m128 x, y, x2, y2, m2;
    __m128i color;

    const simd4f one = _mm_set1_ps(1.0f);
    const simd4f th  = _mm_set1_ps(4.0f);

    x  = _mm_setzero_ps();
    y  = _mm_setzero_ps();
    x2 = _mm_setzero_ps();

```

```

y2 = _mm_setzero_ps();

// iterate f(z) = z^2 + c, z0 = 0
for( int i=0; i < 512; i++ )
{
    y = _mm_mul_ps( x, y );
    y = _mm_add_ps( _mm_add_ps(y,y), b );
    x = _mm_add_ps( _mm_sub_ps(x2,y2), a );

    x2 = _mm_mul_ps( x, x );
    y2 = _mm_mul_ps( y, y );

    m2 = _mm_add_ps(x2,y2);

    // conditional sentence comes here
    // ...
}

// create color
// color = ...
return color;
}

```

Now, how to handle the conditional... Normally, as shown in the original C code, we should skip the iteration loop once the condition $m2 > 4.0f$ becomes true. However, in the SIMD version, there is a small issue, because we are iterating four values at the same time. So, the condition could be true for one, two or even three of the values of $m2$, but not for the others. In that case, we still have to keep on iterating even if results for those values that already fulfilled the condition should not be taken into account. This is a pity, because we will probably waste some computational power for nothing... This is an issue for all SIMD implementation of non-trivial algorithms, but can be alleviated in most cases by preparing the data to the algorithm in such a way that the four data values follow the same path along the conditionals, or at least follow the most similar paths possible. In this our test, it is the case most of the times, since we are iterating together values from the C plane of the Mandelbrot set that are close to each other (the pixels are just contiguous in screen space), thus most of the time the number of iterations before escaping the conditional $m2 > 4.0f$ will be the same. That way, we will hopefully be close to the speed up factor of four.

Then, let's see how we implement the conditional, and the redundant-but-not-disturbing execution of code. First, it's obvious that we can exit the loop once the four iterated values make the conditional true. Thus, every iteration not only we have to perform the comparison itself, but we also have to keep memory of the values that already met the condition (note: this step could be omitted in the case of the Mandelbrot set, since once a value becomes bigger than the escape radius, it will never be smaller again, and thus it will always make the condition be true, but this is not the regular case for other algorithms). Since the comparison instructions in

SSE creates a mask of four values (one per compared value) equal to 0 (false) or 0xffffffff (true), we can use a binary *or* operation to keep the memory of the comparisons. That way, once one of the four conditionals becomes true, it will stay always to true, while it will stay to false until the condition is true for the first time. So, we initialize a variable to *false* (zero), and then, for every iteration, we do the comparison and the *or* operation.

```
// in the begining of the function
__m128 co;

// in initialization (before the loop)
co = _mm_setzero_ps();

// during iteration
co = _mm_or_ps( co, _mm_cmpgt_ps( m2, th ) );
```

Now, we can implement the "optimization" of breaking the loop once all the four values met the condition. We can do that with the `_mm_movemask_ps()` instruction:

```
// during iteration, in the end of the loop
if( _mm_movemask_ps( co )==0x0f )
    break;
```

At this point we are almost done. But we still have to solve a small problem. If you carefully follow the logic of the small algorithm, you will notice that the results from the iterations are not correct, because the last of the four values to fulfill the condition will drive coloring (based on the *i* variable). Somehow we should be able to track the last iteration (value of the *i* variable) at the time each of the four initial values was still active in the 4-way iteration process. An easy solution for that is to create another variable, initialized to zero, that increments by one in each iteration as long as the conditional is false. We can easily do a conditional increment based on the fact that we have the conditional value, doing something like this:

```
// in the begining of the function
__m128 ite

// in initialization (before the loop)
ite = _mm_setzero_ps();

// during iteration, in the end of the loop
ite = _mm_add_ps( ite, _mm_andnot_ps( co, one ) );
```

Now, when the loop execution finishes, we have the correct number of iterations each of the four values needed to met the condition. Now, we just have to build the four output colors based in these values as we did in the scalar version of the algorithm. Here goes the complete SSE version:

```

__m128i IterateMandelbrot( __m128 a, __m128 b )
{
    __m128  x, y, x2, y2, m2;
    __m128  co, ite;

    unsigned int i;

    const simd4f one = _mm_set1_ps(1.0f);
    const simd4f th  = _mm_set1_ps(4.0f);

    x  = _mm_setzero_ps();
    y  = _mm_setzero_ps();
    x2 = _mm_setzero_ps();
    y2 = _mm_setzero_ps();
    co = _mm_setzero_ps();
    ite = _mm_setzero_ps();

    // iterate  $f(Z) = Z^2 + C$ ,  $Z_0 = 0$ 
    for( i=0; i < 512; i++ )
    {
        y = _mm_mul_ps( x, y );
        y = _mm_add_ps( _mm_add_ps(y,y), b );
        x = _mm_add_ps( _mm_sub_ps(x2,y2), a );

        x2 = _mm_mul_ps( x, x );
        y2 = _mm_mul_ps( y, y );

        m2 = _mm_add_ps(x2,y2);
        co = _mm_or_ps( co, _mm_cmpgt_ps( m2, th ) );

        ite = _mm_add_ps( ite, _mm_andnot_ps( co, one ) );
        if( _mm_movemask_ps( co )==0x0f )
            i=nites;
    }

    // create color
    const __m128i bb = _mm_cvtps_epi32( ite );
    const __m128i gg = _mm_slli_si128( bb, 1 );
    const __m128i rr = _mm_slli_si128( bb, 2 );
    const __m128i color = _mm_or_si128( _mm_or_si128(rr,gg),bb );

    return( color );
}

```

The result is that by using data parallelizable version of an algorithm, we can increase the performance by a factor of three (if not four) by just slightly modifying the code. For that, additional code has to be written to handle the different paths the algorithm could take for the four data inputs. However, by using inter-coherent

data input, this effect can be reduced dramatically in most cases and get close to the theoretical performance speed up announced by CPU vendors.

I used the code above to create [this executable](#), that showed a speed up of three compared to the normal C code.

[inigo quilez](#) - learning computer graphics since 1994